Memory Corruption
Taller de Seguridad
Ing. Martin Di Paola



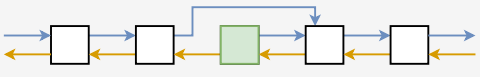
double
linked list



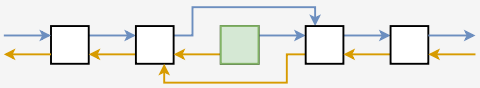
double
linked list

```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```

Supone que se quiere eliminar un nodo de la lista doblemente enlazada. Que harias?




```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```




```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```




```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```



double
linked list

what do
you see?

```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```

Que ves?



double
linked list

what do
you see?

a remove-node-
from-list primitive?

```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```

Que ves? Código para remover un nodo de una lista? Un método/función interna de un TDA o clase?



double
linked list

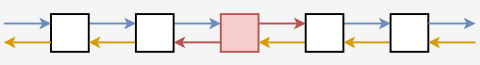
what do
you see?

a remove-node-
from-list primitive?

an arbitrary write
primitive!

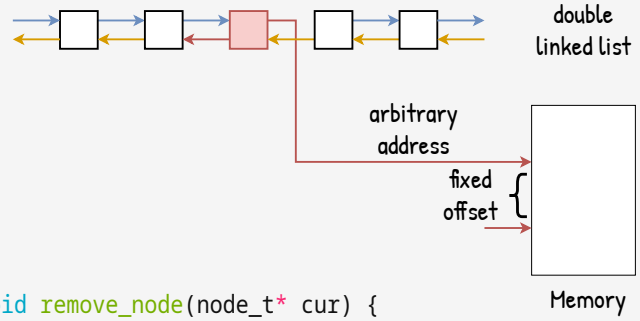
```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```

Un hacker ve una primitiva de escritura arbitraria.



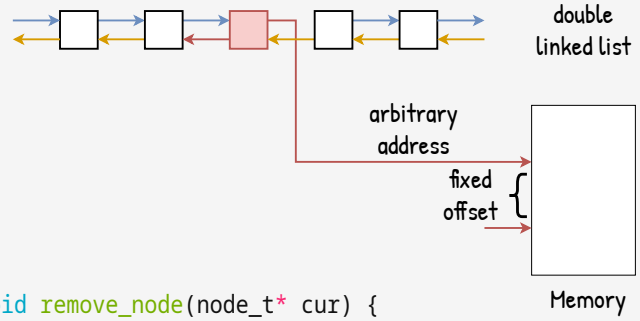

```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```

Supone que un atacante tiene control del nodo. No solo del "dato" sino también de los punteros siguiente y anterior (next / prev)




```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```

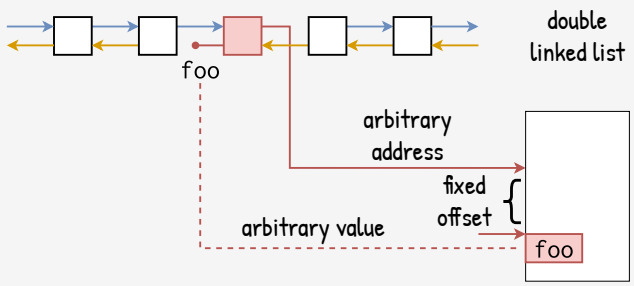
En vez del puntero next apuntar a un nodo, este puede apuntar a cualquier parte de la memoria.




```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```

Ahora, el método/función que remueve un nodo trabaja para el atacante.

La idea es que cuando se haga `cur->next->prev` se va a estar accediendo a cualquier memoria seleccionada por el atacante (`cur->next`) más un offset fijo debido al acceso del atributo `prev` en `cur->next->prev`

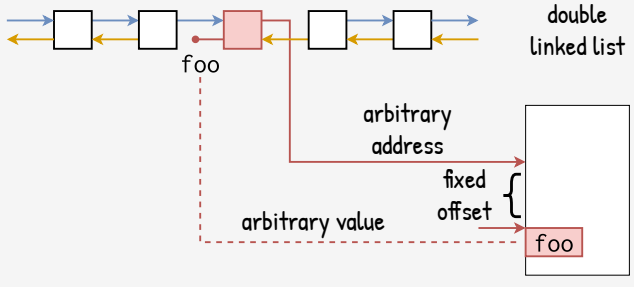



```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```

Y que valor se va a escribir? Pues `cur->prev` que está también bajo el control del atacante!

`cur->prev` es un puntero pero en el fondo son 8 bytes con datos. El atacante puede guardar lo que quiera y por ende escribir lo que quiera.

Una primitiva de escritura arbitraria.




```
void remove_node(node_t* cur) {
  cur->prev->next = cur->next;
  cur->next->prev = cur->prev;
}
```

unwanted side effect

Pero todo tiene un catch. Manipular los punteros de un nodo para tener un efecto específico puede tener efectos indeseados.

Esto casi seguro termina en un crash. La magia esta en poder tomar control de un sistema antes que crashee!

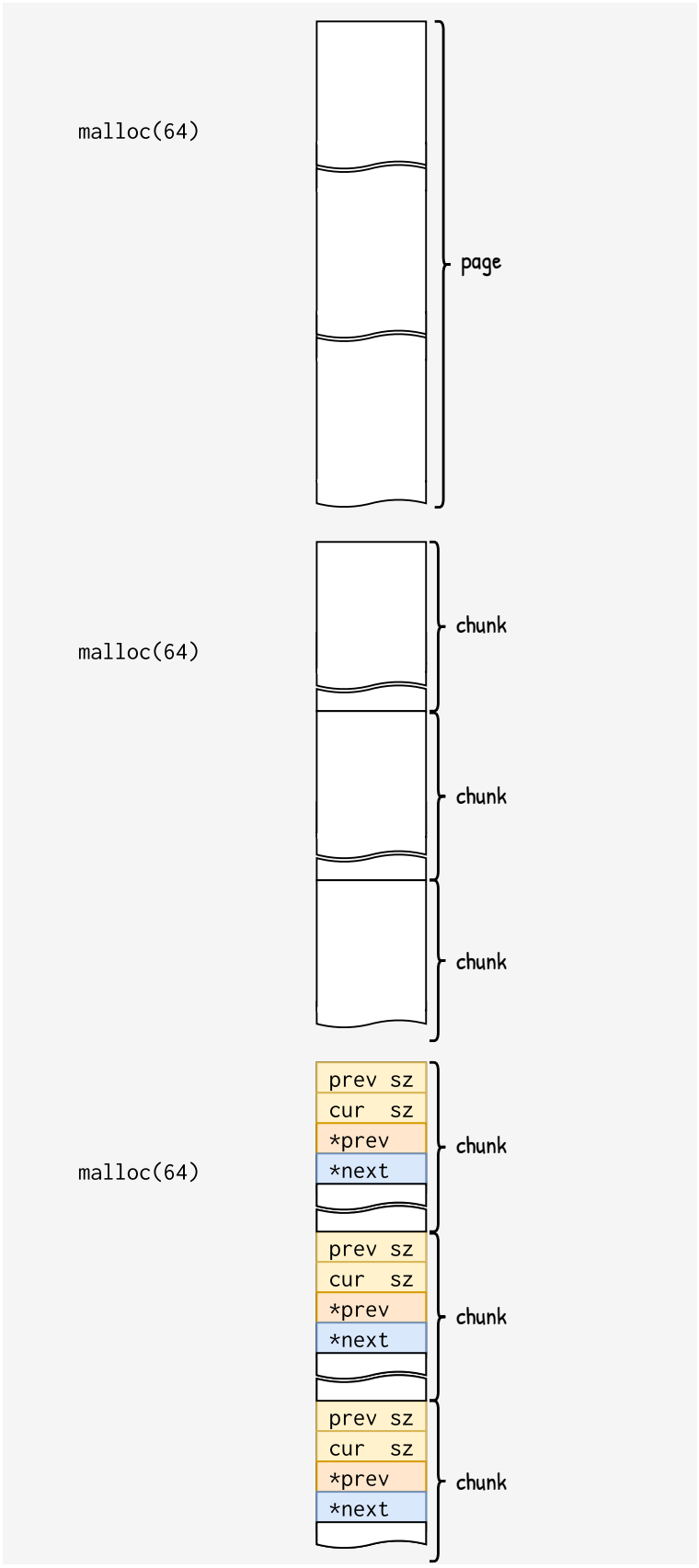
```
malloc(64)
```

Como funciona malloc? A alto nivel uno diria que el programa le pide memoria al kernel pero es mucho más complejo que esto.

Entre la aplicación y el kernel esta el memory manager, una librería del mismo programa que ofrece la implementación de malloc, realloc y free (y alguna q otra cosilla mas).

Hay múltiples memory allocators. En linux, usando la GNU libc, se usa un derivado del allocator de Doug Lea creado a fines de los 80s. Viejo pero funciona muy bien.

Vamos a ver un modelo simplificado de él.



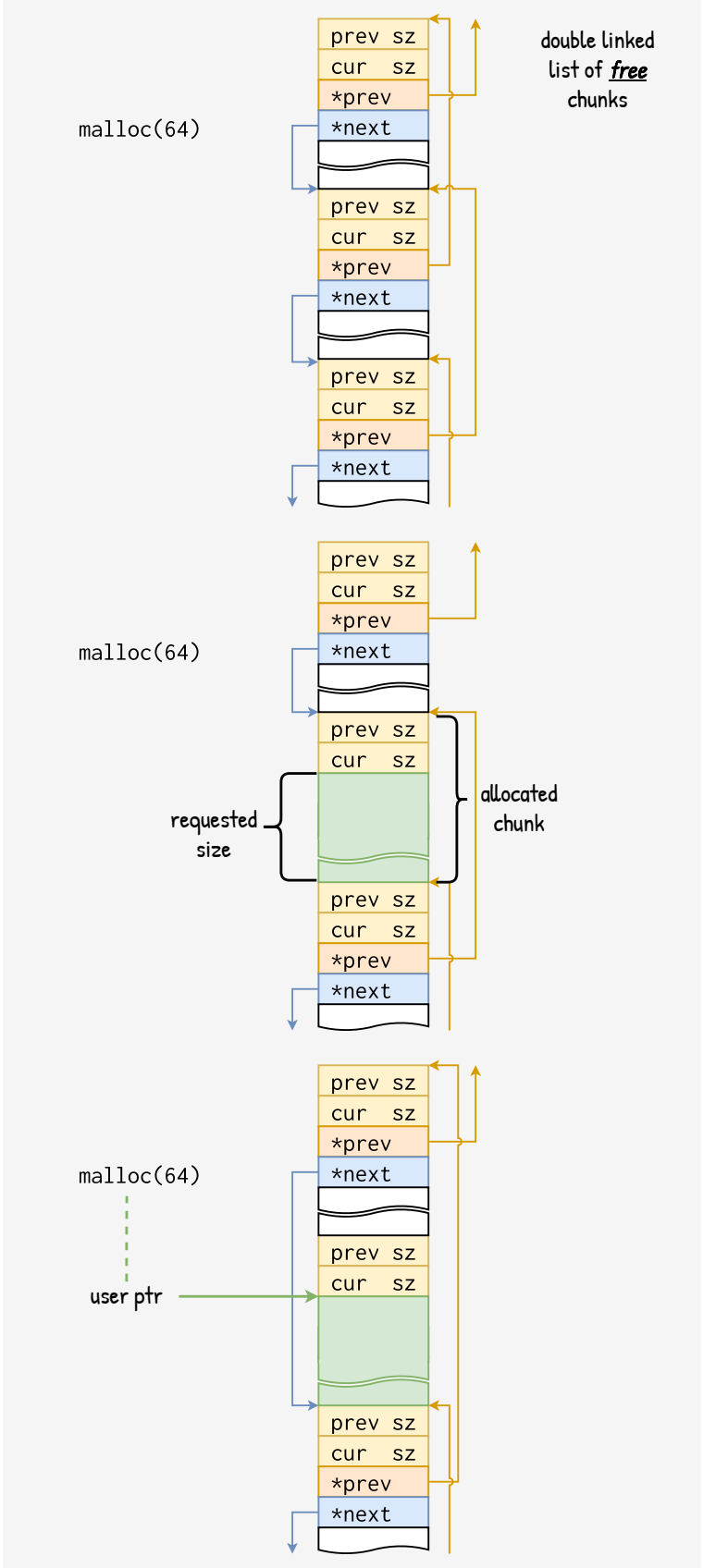
Ante un request de memoria, el allocator va a buscar memoria libre. Si no la encuentre le pide memoria al kernel.

El kernel no entrega "un par de bytes" sino chunks mas grandes. En linux, estos se llaman páginas.

El memory allocator particiona la pagina en unidades más chicas llamadas chunks.

Cada chunk tiene metadata: los sizes del chunk actual y anterior y unos punteros.

En la implementacion de GNU (y en otras tambien) esta metadata se escriben dentro del chunk.



Los punteros forman una lista doblemente enlazada de chunks libres.

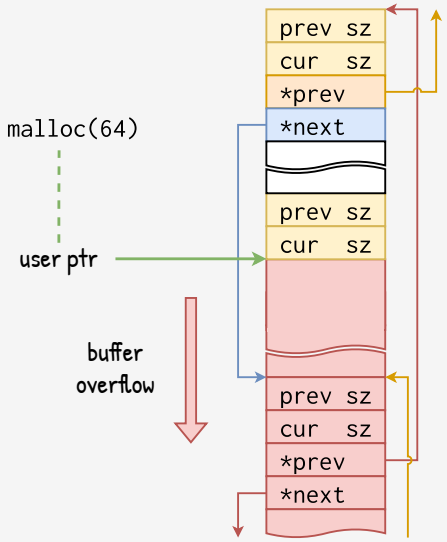
Para completar el request de memoria, el allocator buscan en la lista de chunks libres alguno para reservar.

El allocator va a tratar de encontrar uno del tamaño más adeucado para la memoria pedida para no desperdiciar.

El allocator de GNU tiene pre-clasificados los chunks libres en cajas o “bins”. Asi cuando busca chunks chicos solo busca en la caja de chunks chicos y no en todo el resto.

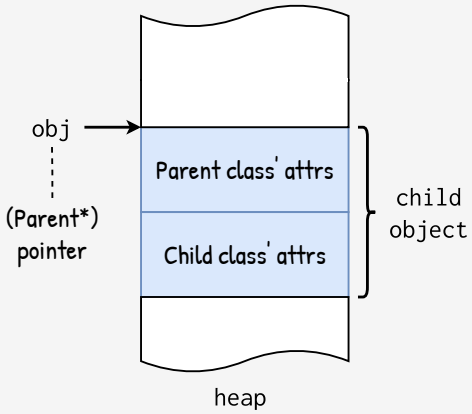
Una vez encontrado un chunk libre adecuado este se remueve de la lista de libres y se le retorna un puntero al user.

Notar que parte de la metadata (los punteros) no se preservan mientras que otra parte (los sizes) si.



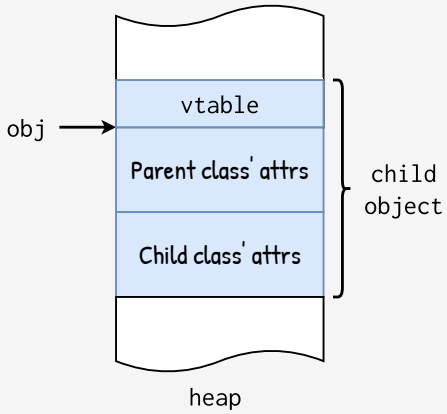
Ahora un adversario va a tratar de conseguir un buffer overflow.

Con él, no se pisa el stack sino el heap y asi puede sobreescribir la metadata (en particular los punteros) y obligar al memory allocator a que haga writes arbitrarios.



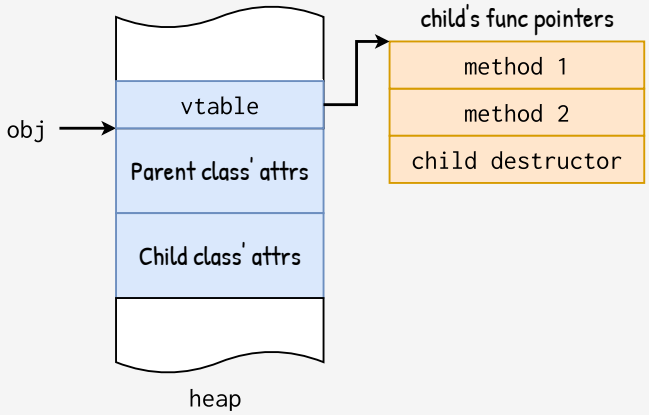
Supongamos que hay un objeto en C++ de tipo Child. Este hereda de la clase padre Parent.

En memoria, un puntero a un objeto Child (aun viendolo como un puntero a Parent) apunta a los atributos de Parent seguidos de los atributos de Child.

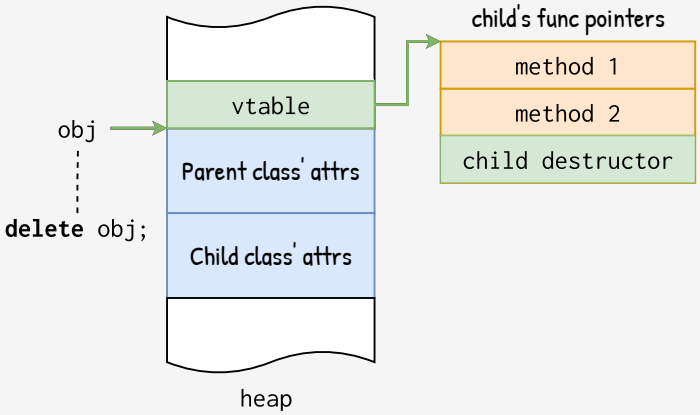


Pero esto no es todo. Para los objetos polimórficos hay un puntero adicional a la virtual table (vtable).

Notar que el puntero que el user ve, no apunta a la vtable sino al principio de los atributos. La vtable está 8 bytes atras.

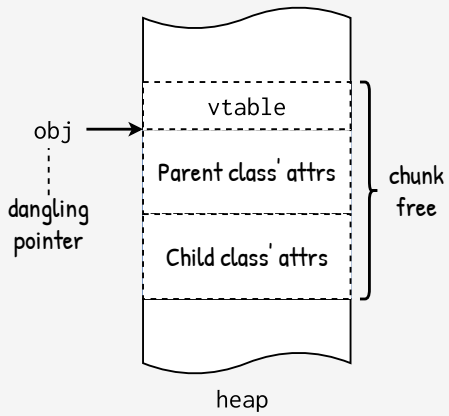


La virtual table es una tabla de puntero a funciones (a métodos). Es una tabla por cada clase polimórfica y no por objeto; todos los objetos de la misma clase van a tener cada un su propio puntero todos apuntando a la misma tabla.

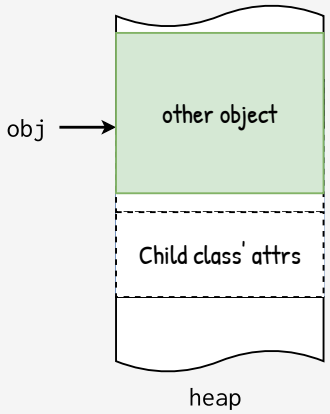


Al hacer `delete` se invoca al destructor de la clase. Aun siendo un puntero a Parent, el destructor es polimórfico por lo que el código no salta al destructor de Parent sino que sigue la `vtable` y llama a la función correcta (el destructor de Child)

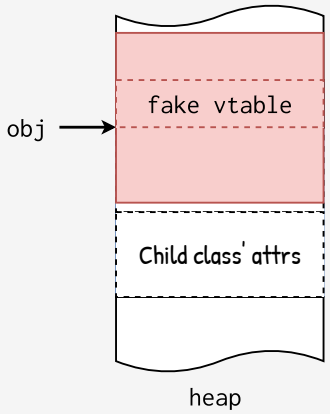
Esto es como C++ implementa el mecanísmo de polimorfismo.



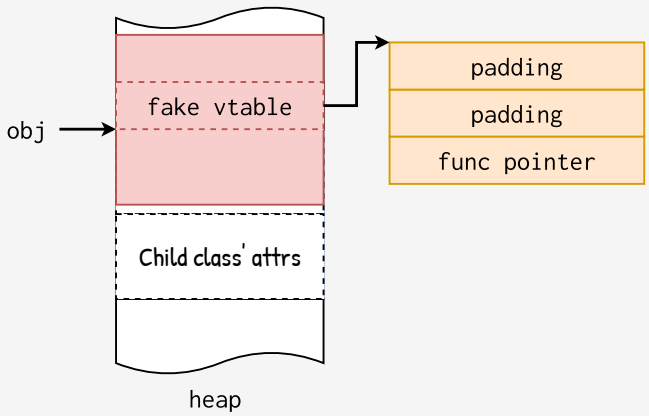
Tras un delete, el puntero se queda “colgado” (dangling). Apunta a la misma región de memoria solo que está no esta reservada.



Dado que la memoria esta libre, el memory allocator puede usarlo (parcial o completamente) para allocar otro objeto.

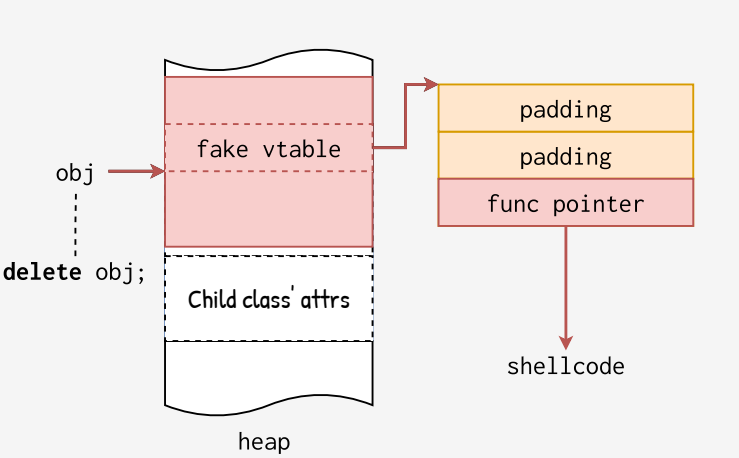


Y si ese alloc, ese objeto lo controlase un atacante? Este podría sobreescribir la vtable del objeto anterior borrado.



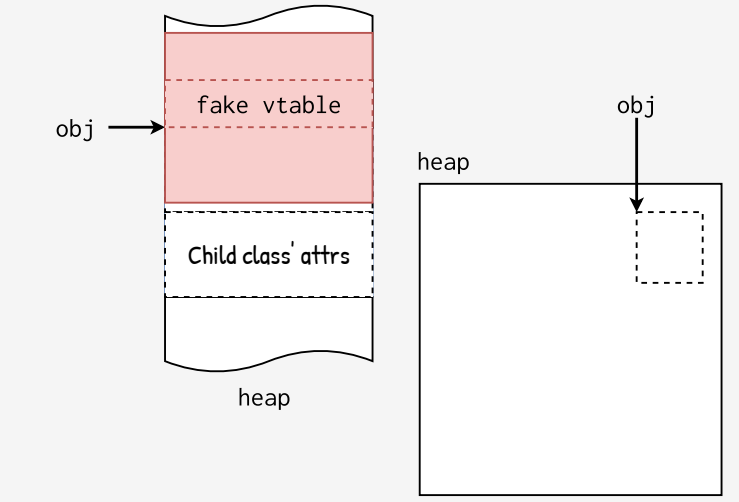
Y con el override, el atacante puede apuntar a una vtable falsa. Si sabe que el código eventualmente va a reutilizar el dangling pointer puede poner punteros a funciones que el desee en la vtable esperando que el codigo lo llame.

La vulnerabilidad no esta en que un atacante puede hacer el override (por q es memoria libre y es totalmente legal escribir en ella) sino en que el programa usa a un dangling pointer -> usa memoria liberada (use after free) o en la jerga “usa memoria fritada”.

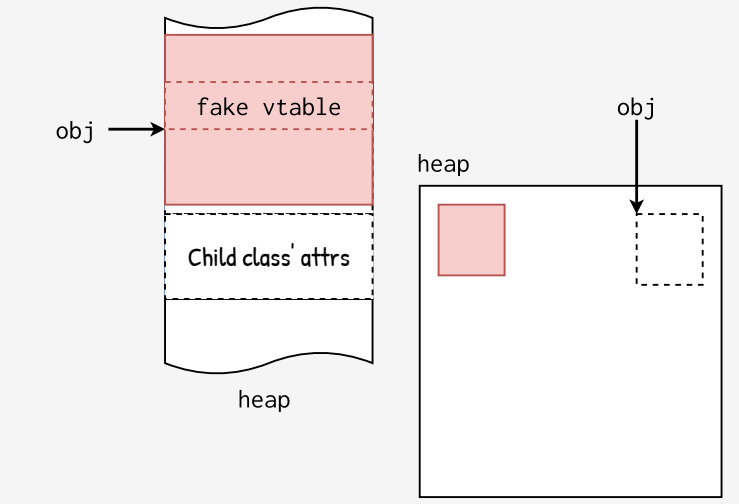


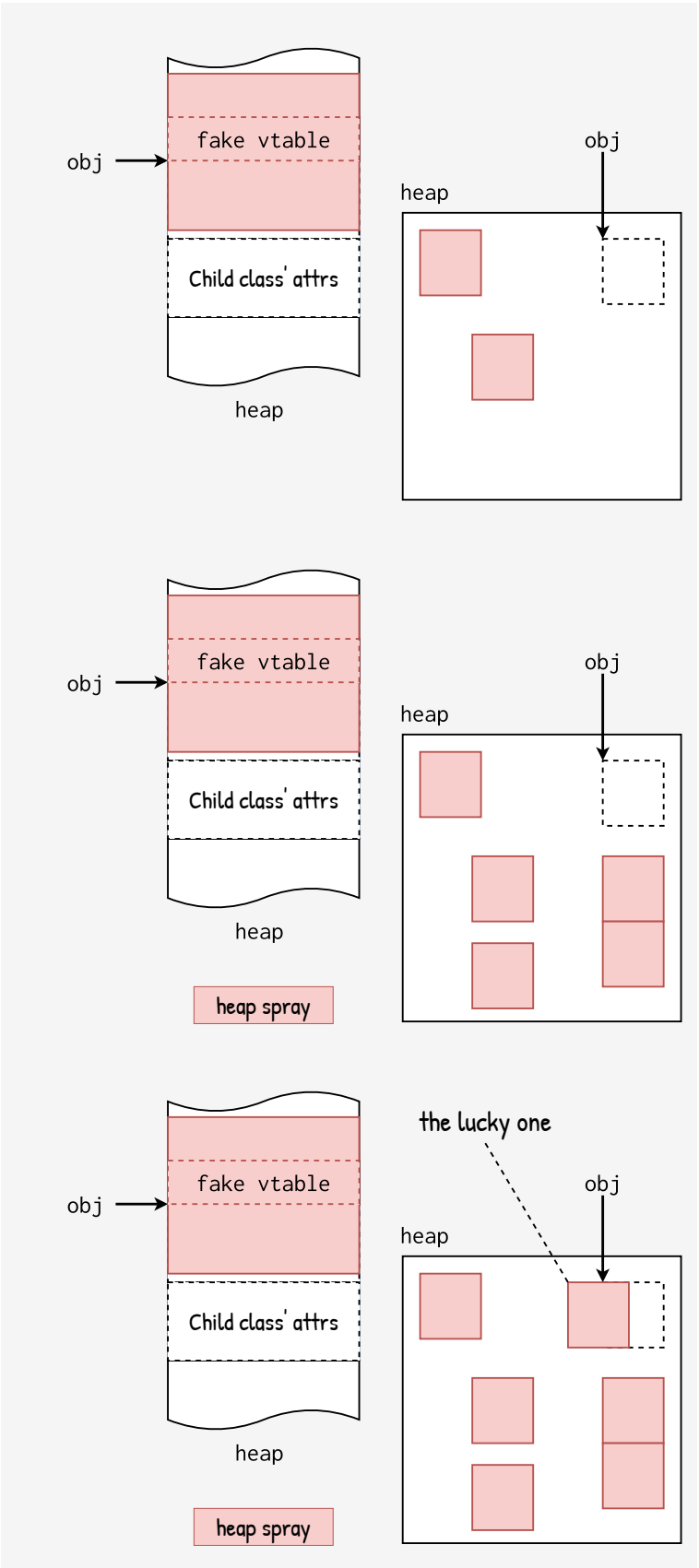
Si la aplicacion comete el error de querer borrar un mismo objeto 2 veces esto se llama double delete (o double free). Es una vulnerabilidad específica del grupo más grande use-after-free.

En general esto termina en un crash pero si el atacante dejó una vtable (falsa) válida, puede hacer q se llame a un destructor falso, código que el atacante desea.



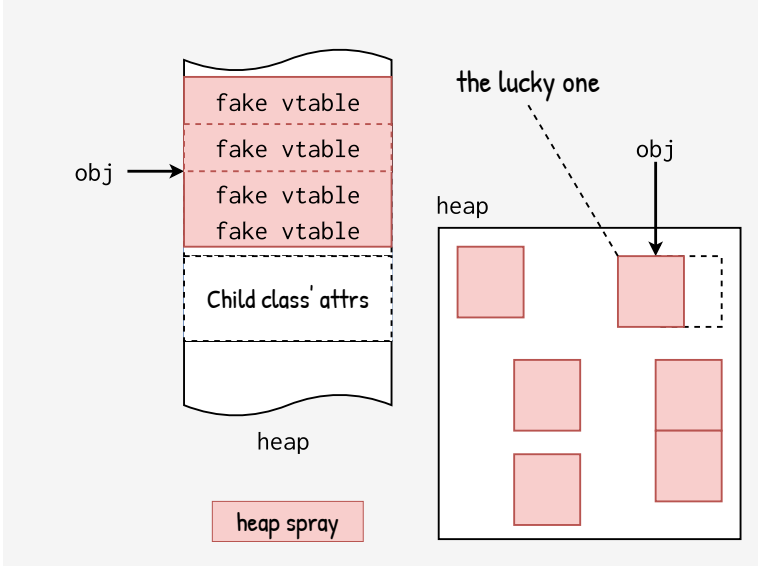
Queda preguntarse, como es que el atacante logra justo reservar memoria justo en el mismo lugar del objeto borrado y pisar justo la vtable?





El truco está en allocar múltiples objetos con la esperanza que uno, de casualidad justo pise el objeto borrado.

Esto se llama heap spray.



Y ahora que el atacante logro pisar la vtable, como sabe en que parte poner el puntero a la vtable falsa?

No lo sabe! asi que sus objetos son un array de punteros todos a la misma tabla falsa. Con algo de suerte el dangling pointer va a quedar apuntando justo luego de la vtable falsa.